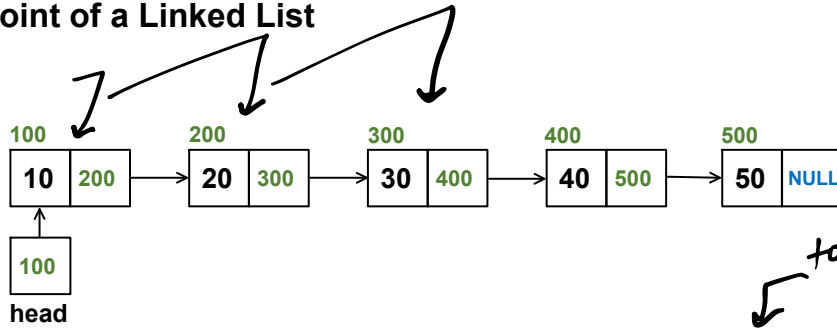


Mid-Point of a Linked List

$$\frac{5}{2} = 2$$

$$1 = 5$$

Mid-Point of a Linked List



to find the len of the LL
to get the mid-point
 $1 + \frac{n}{2} = \frac{3n}{2} \cdot \text{const}$

time! $O(n)$

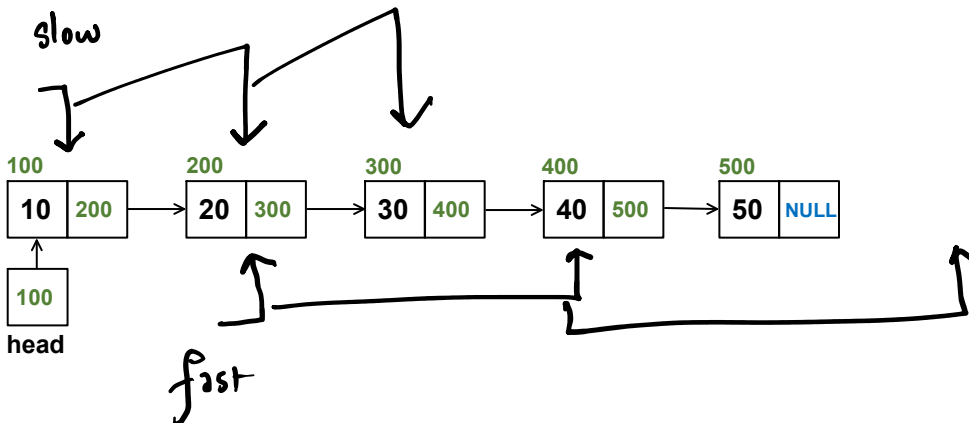
tortoise - hare
(slow) (fast)

⇒ Slow-fast ptr app

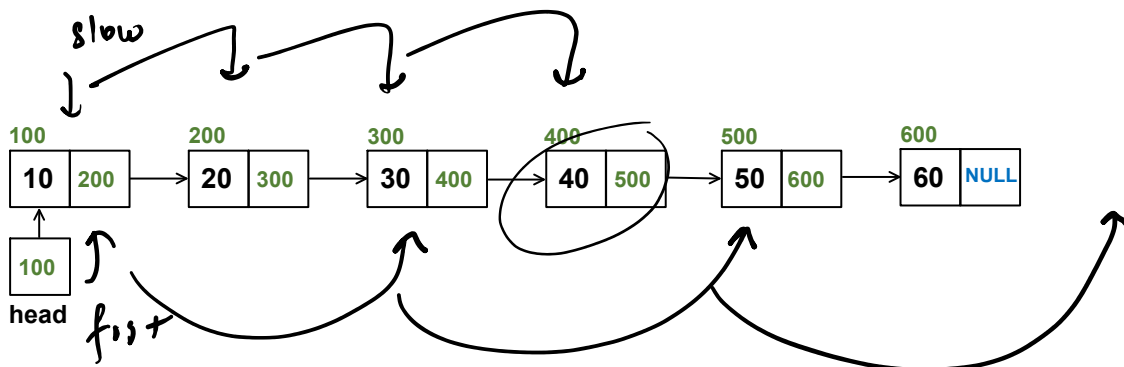
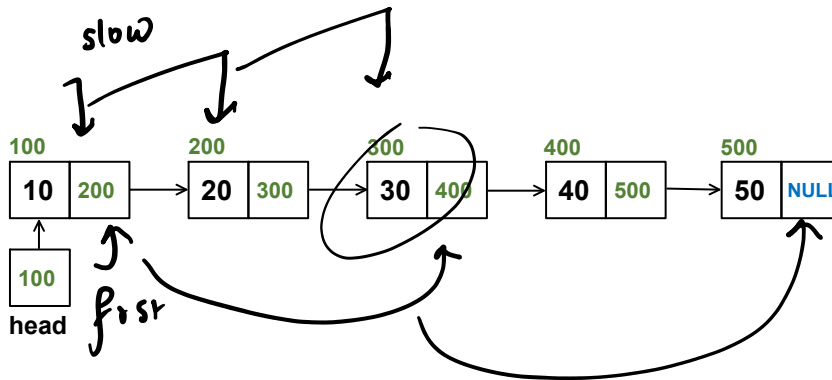
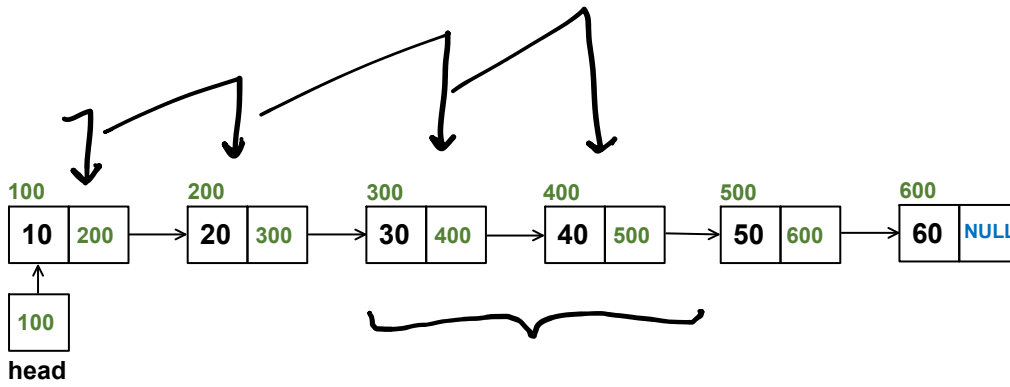
(x kmph)
slow
fast
(2x kmph)

slow

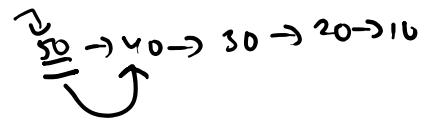
fast



$$\eta = 6 \quad \frac{1}{2} = 3$$

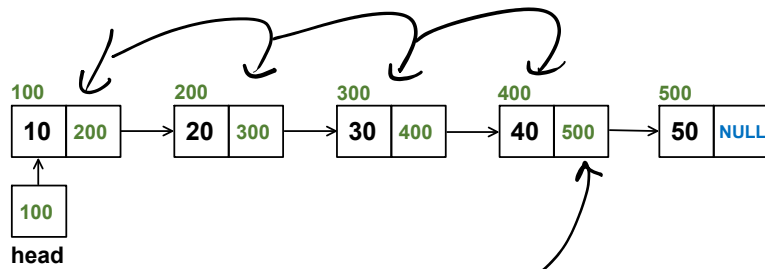


$$n = 5$$



$$K = 2$$

Kth Node from End in a Linked List



Kth last node

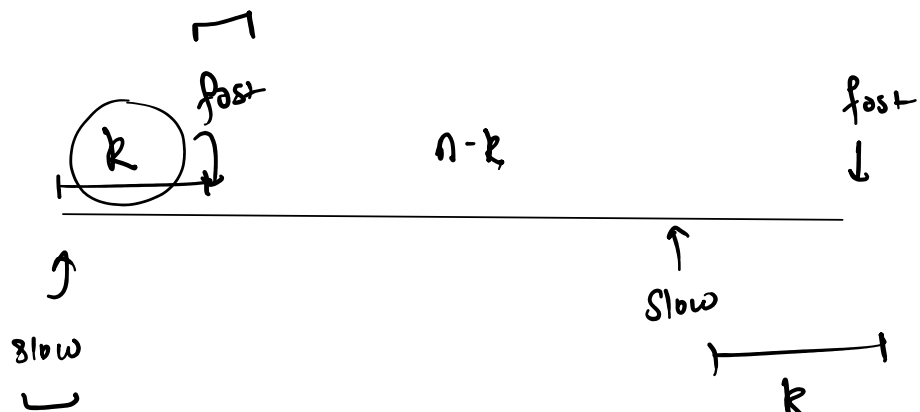
app^r 1

1. find the len of the ll = n
2. Move $n - K$ steps from head = $\frac{n - K}{2n - K}$

app^r 2

1. Reverse the ll = n
2. move $K - 1$ steps from head = $\frac{K - 1}{n + K - 1}$

app^r 3



$$K + (n - K) = n \text{ steps}$$

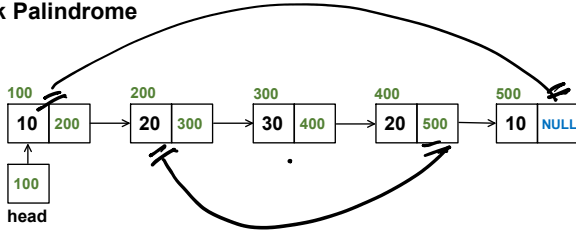
move fast by K-steps
 move slow & fast

move to
by 2-steps
and initialise
slow with
head

move slow & fast
at the same speed
till fast becomes
null, at this point
slow will be at the
kth last node

Check Palindrome

i/p:



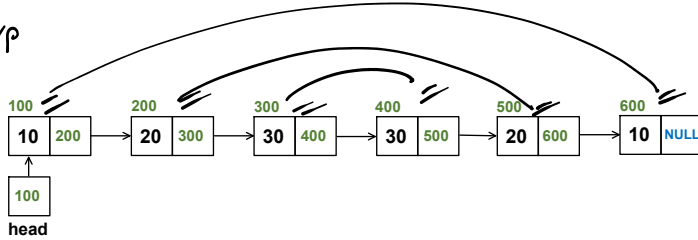
- (two)
1. create a clone of given LL $\Rightarrow n$
 2. reverse the clone $\Rightarrow n$
 3. cmp with org. ll $\Rightarrow n$
- 3n steps

time: $O(n)$

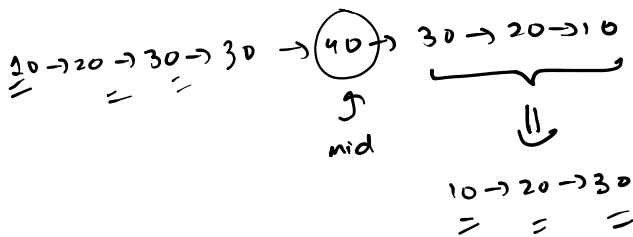
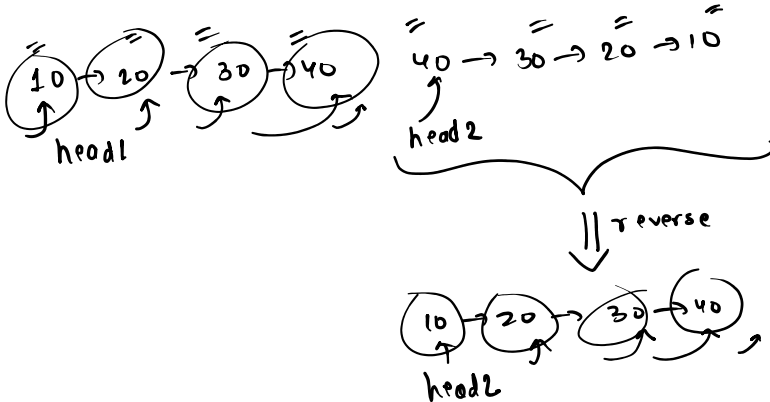
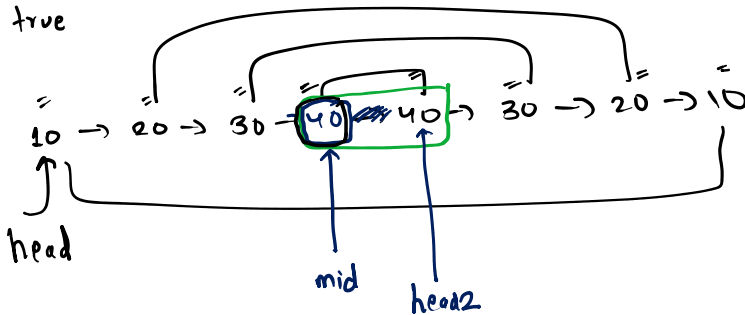
space: $O(n)$ due to clone

o/p: true

i/p

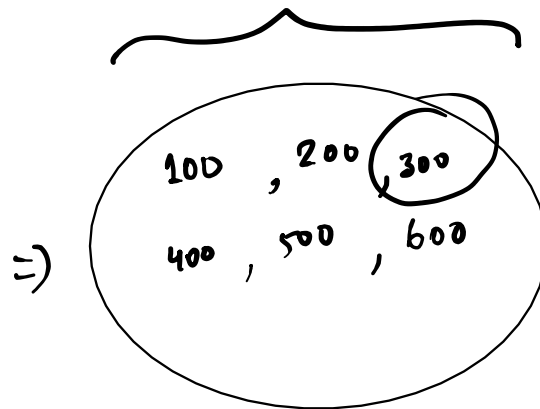
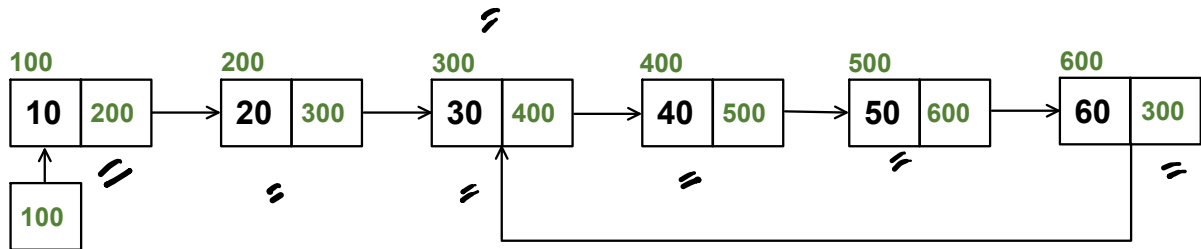


o/p: true



Detect Cycle in a Linked List

Detect Cycle in a Linked List

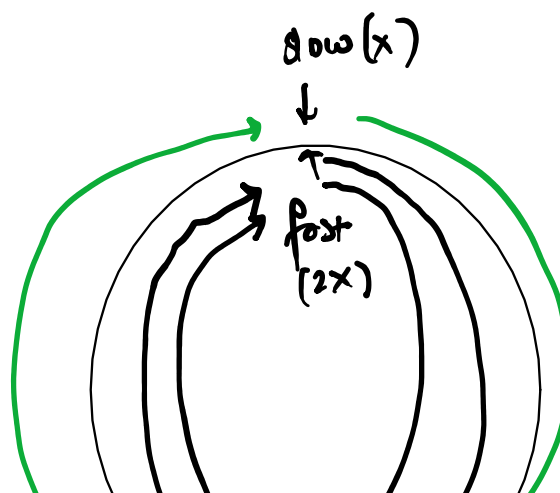


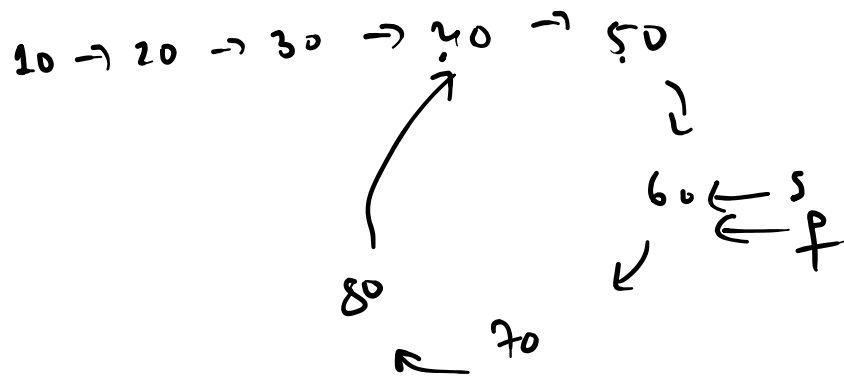
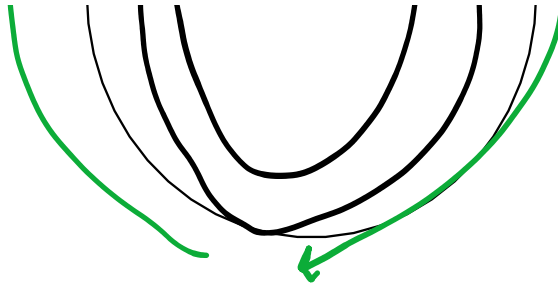
(x)
slow
↓

slow
↓

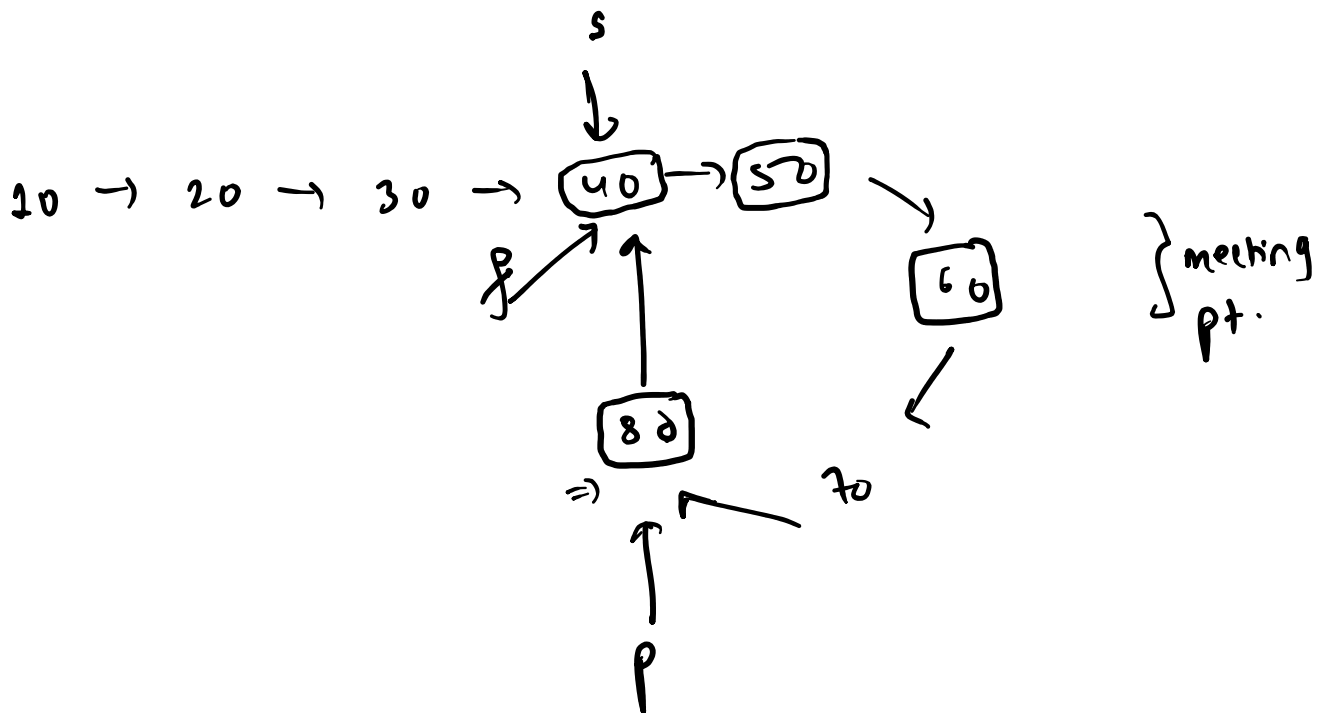
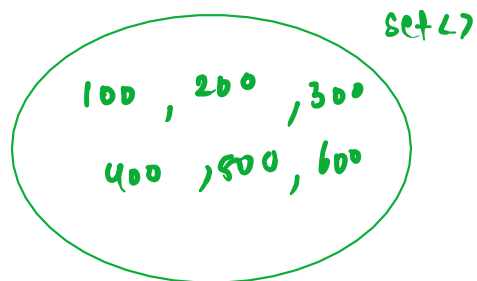
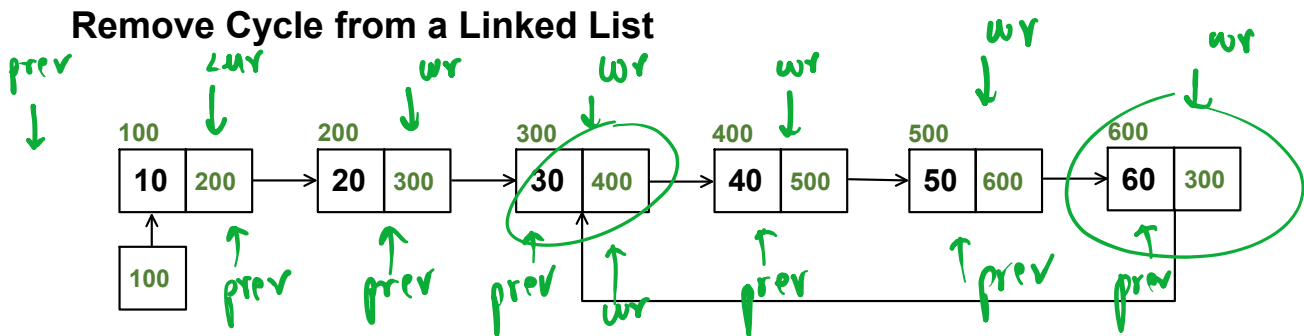
↑
fast
(2x)

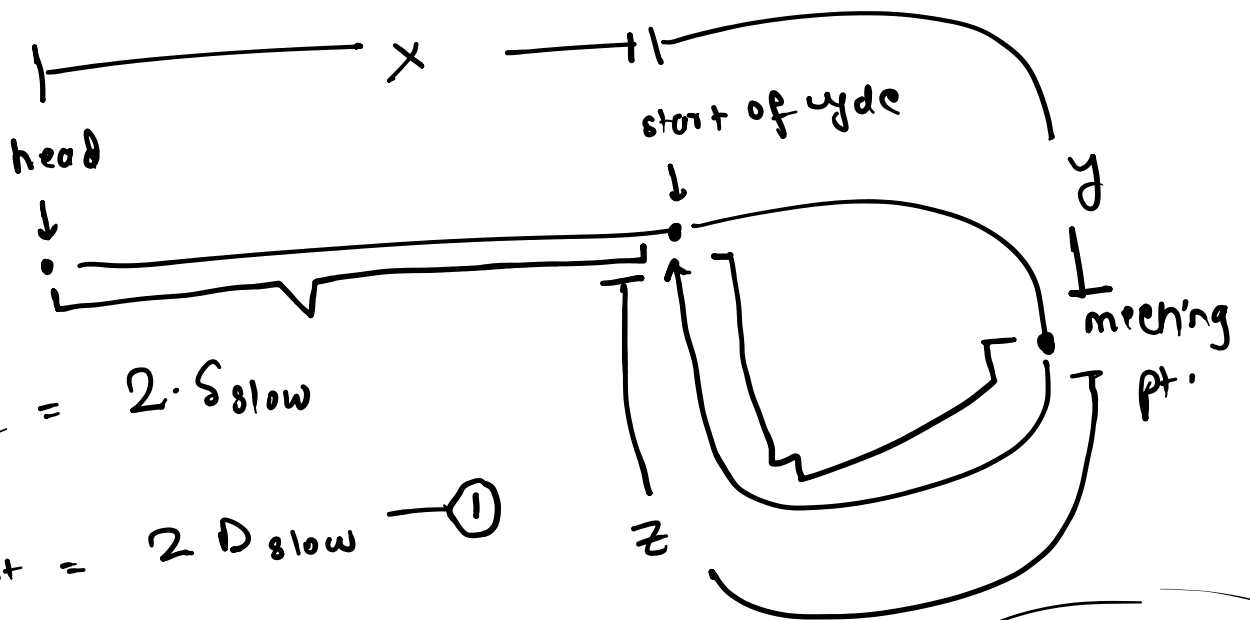
↑
fast
=





Remove Cycle from a Linked List





$$S_{fast} = 2 \cdot S_{slow}$$

$$\Rightarrow D_{fast} = 2 D_{slow} \quad \textcircled{1}$$

$$D_{fast} = x + y + z + y = x + 2y + z$$

$$D_{slow} = x + y$$

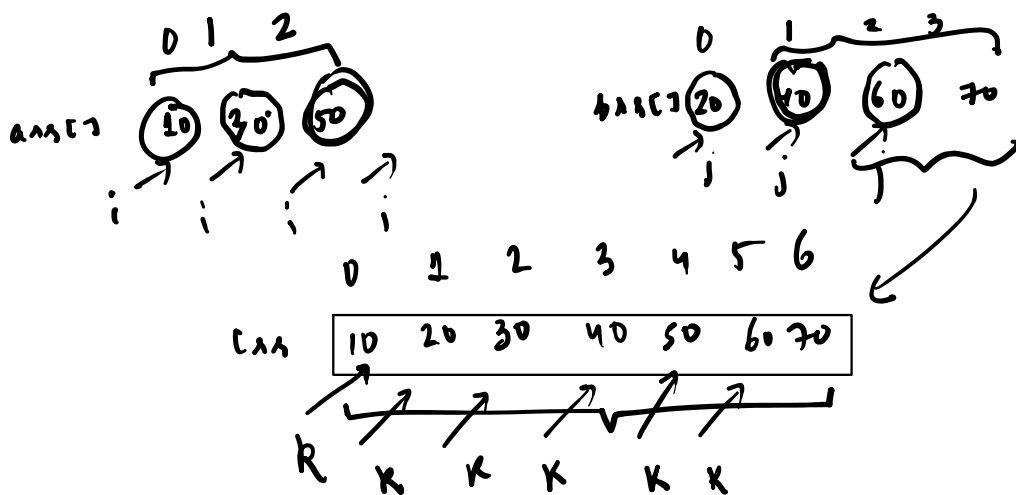
$$x + 2y + z = 2(x + y)$$

$$\cancel{x} + \cancel{2y} + z = \cancel{2x} + \cancel{2y}$$

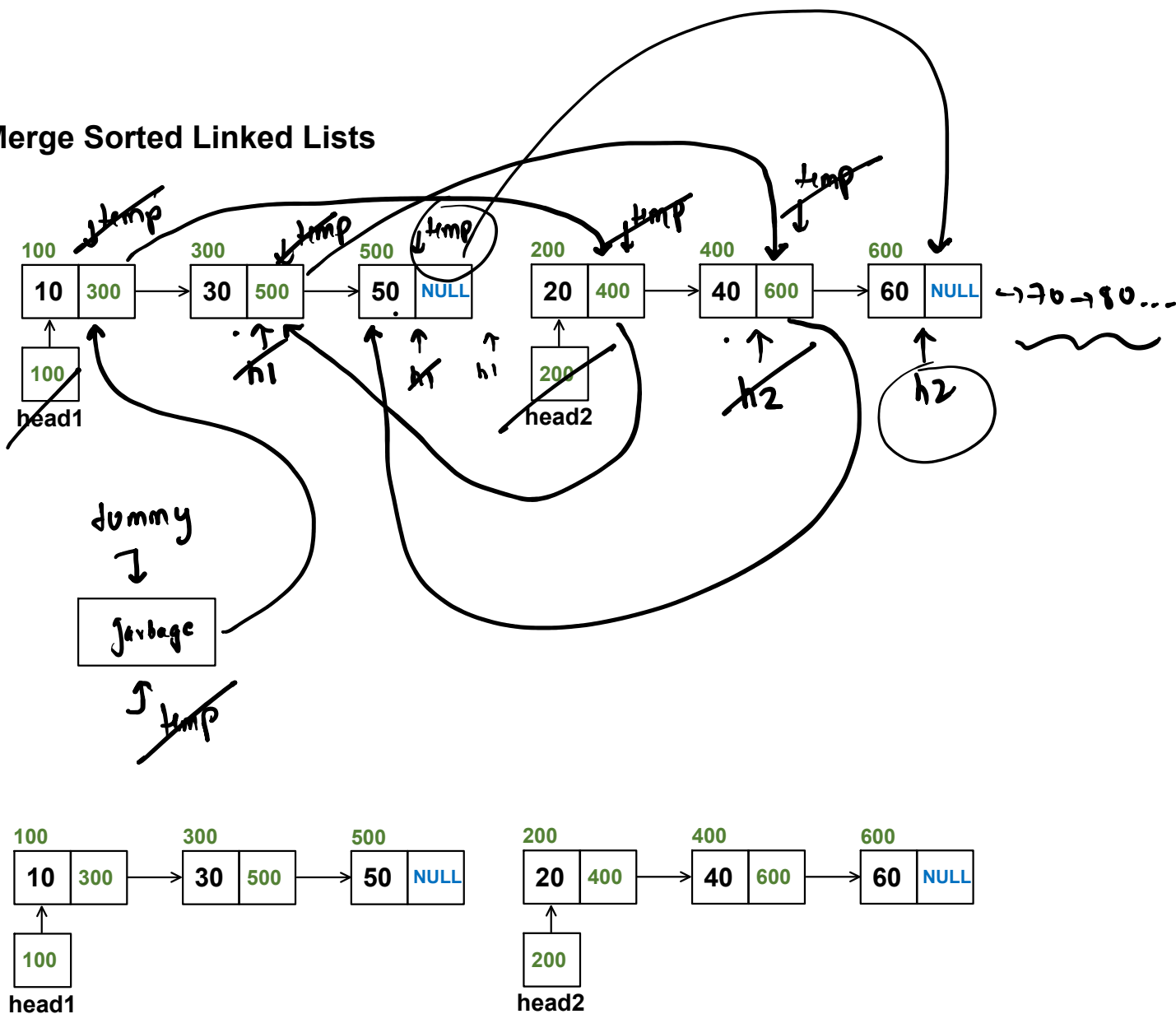
$$\boxed{z = x}$$

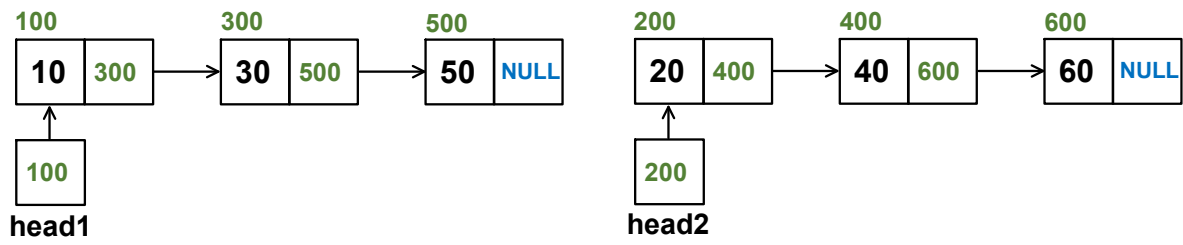
Merge Sorted Linked Lists

[HW] try to solve this question recursively

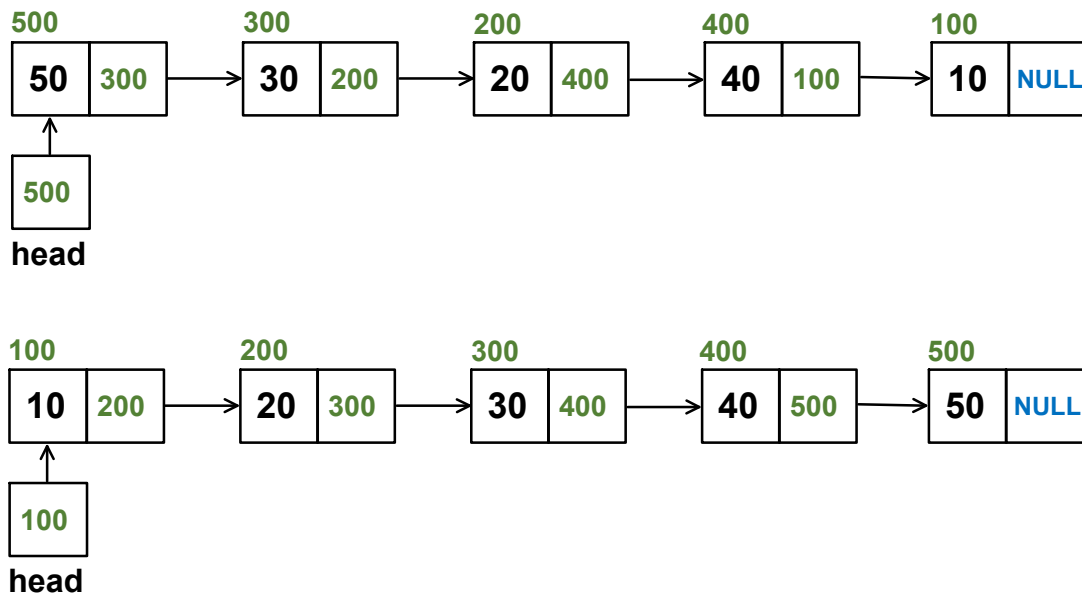


Merge Sorted Linked Lists



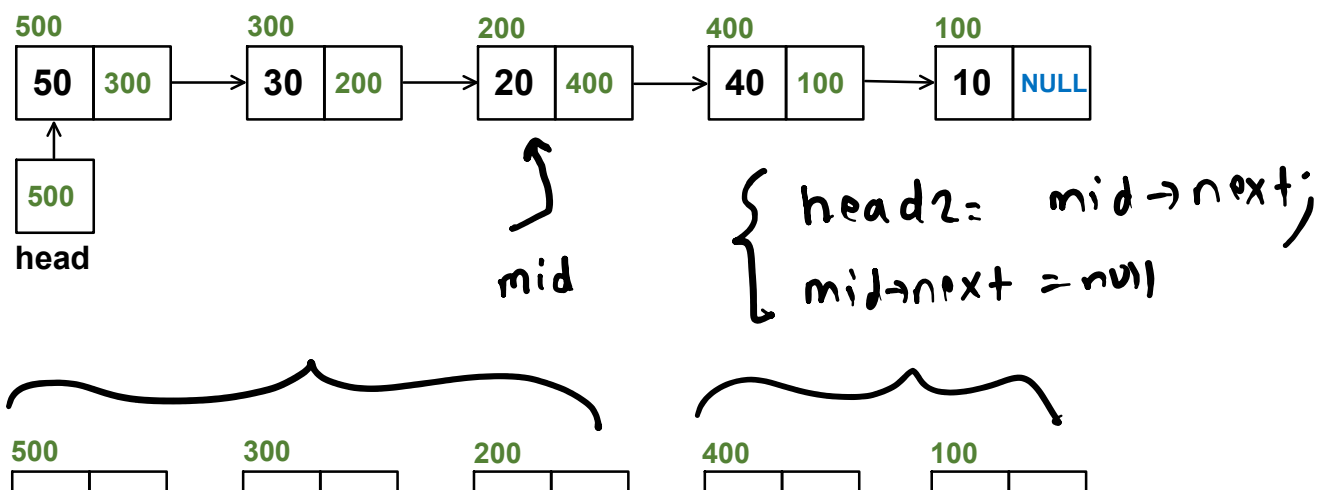


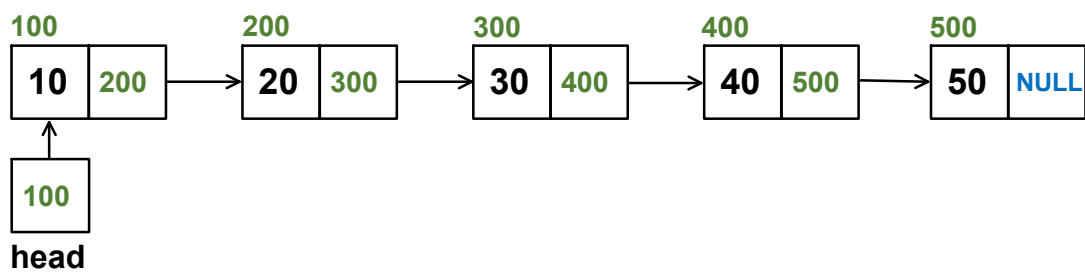
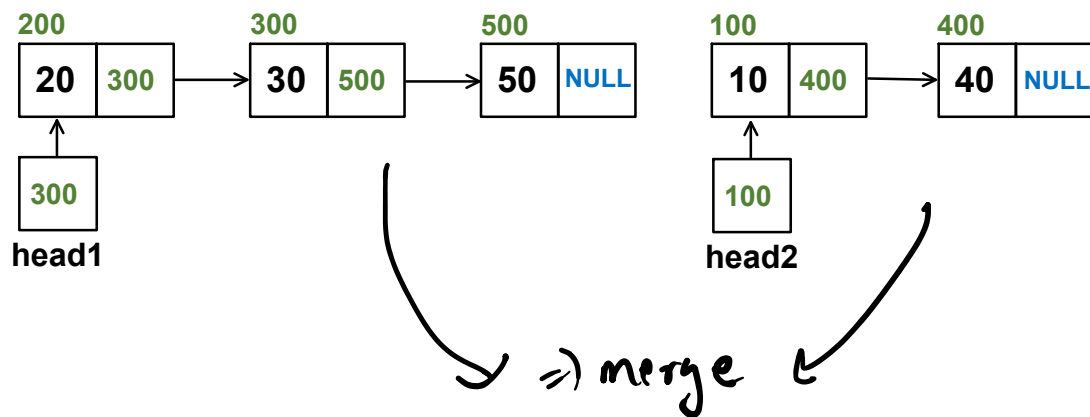
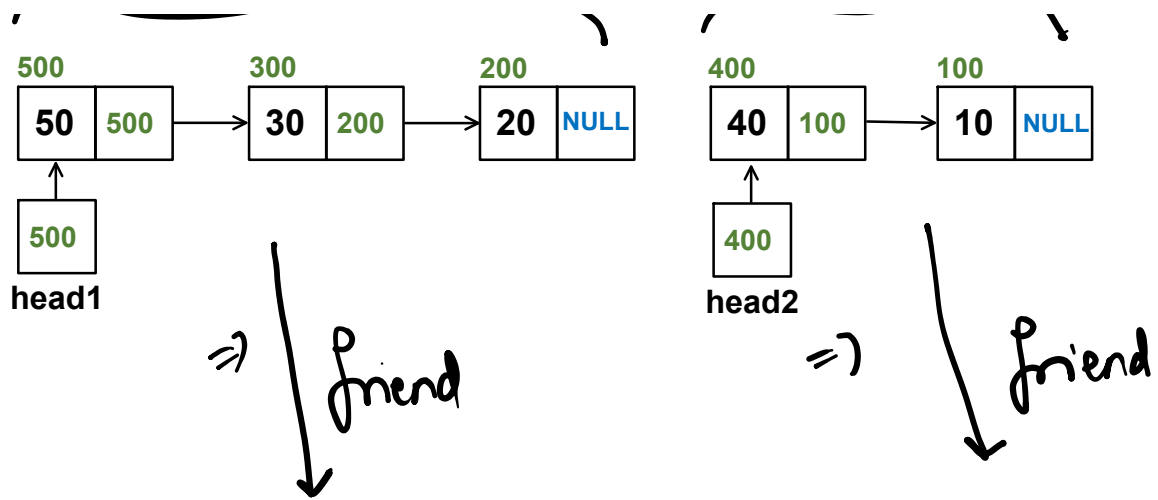
Merge-Sort a Linked List



To **sort** a linked list using the **merge-sort** algorithm we follow three steps

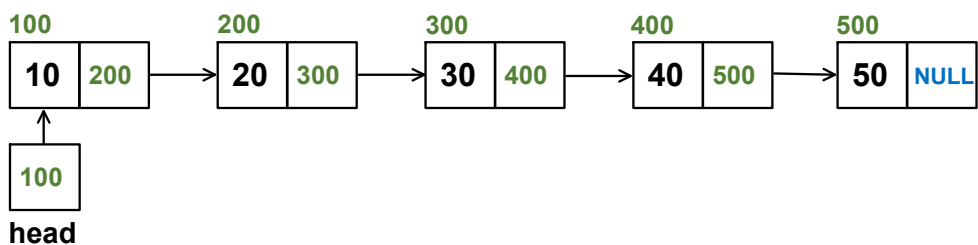
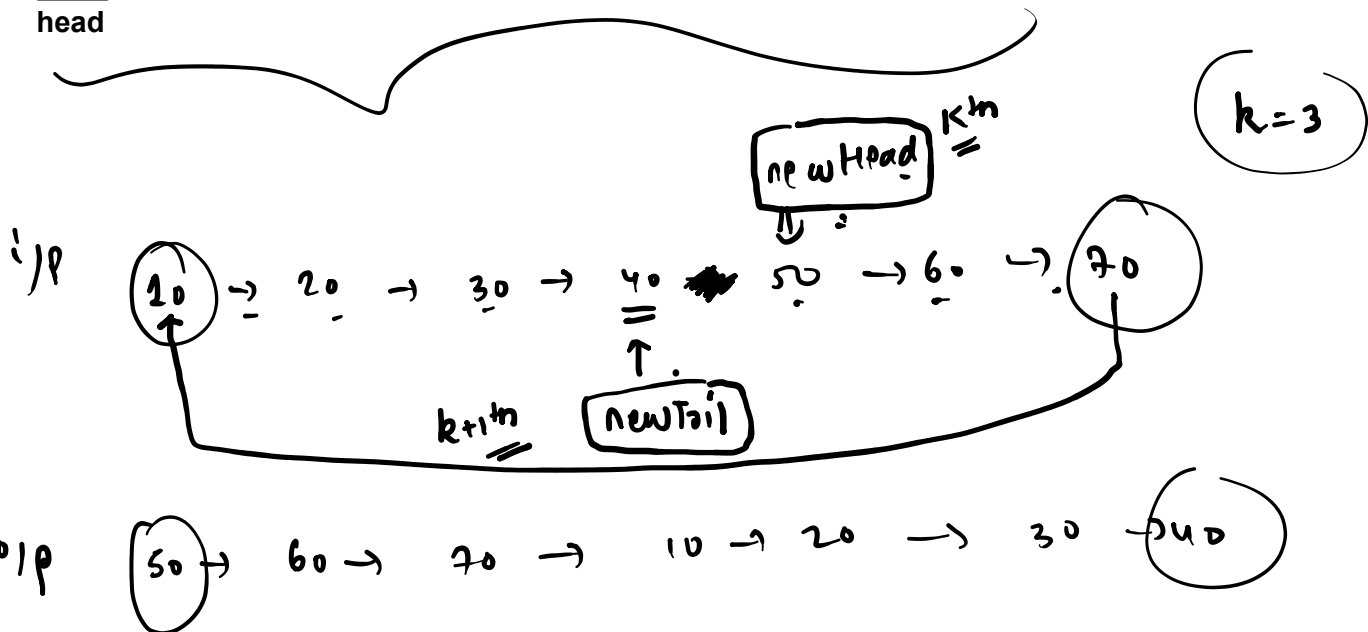
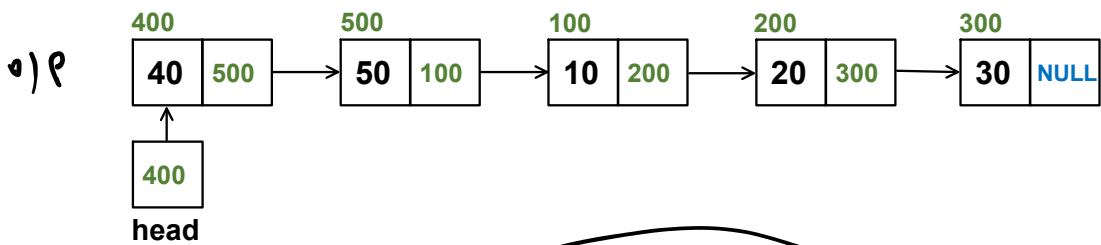
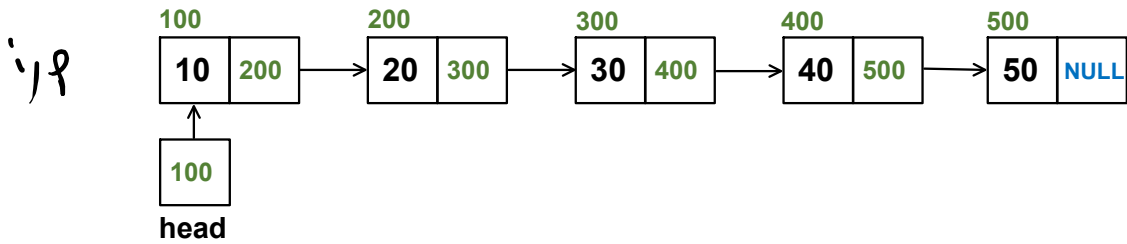
- ⇒ ○ **divide** the linked list into two sub-lists around the mid-point
- ⇒ ○ **recursively** sort the two sub-lists
- **merge** the two sorted sub-lists such that the merged list is sorted.





K-Rotate a Linked List

$k=2$



[R][HW] Sum of Linked List

=

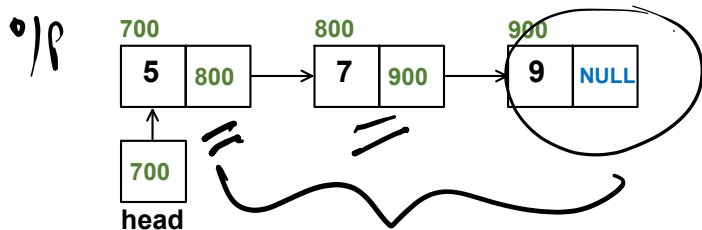
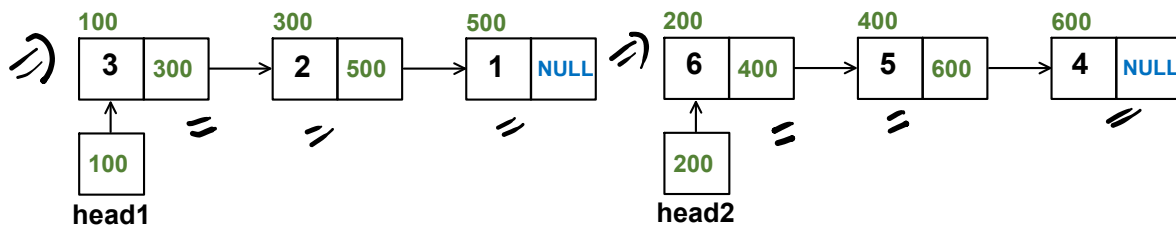
→ $n_1 =$ 123
→ $n_2 =$ 456

1 → 2 → 3
4 → 5 → 6

↓ ↓ ↓
1 2 3
+ 4 5 6

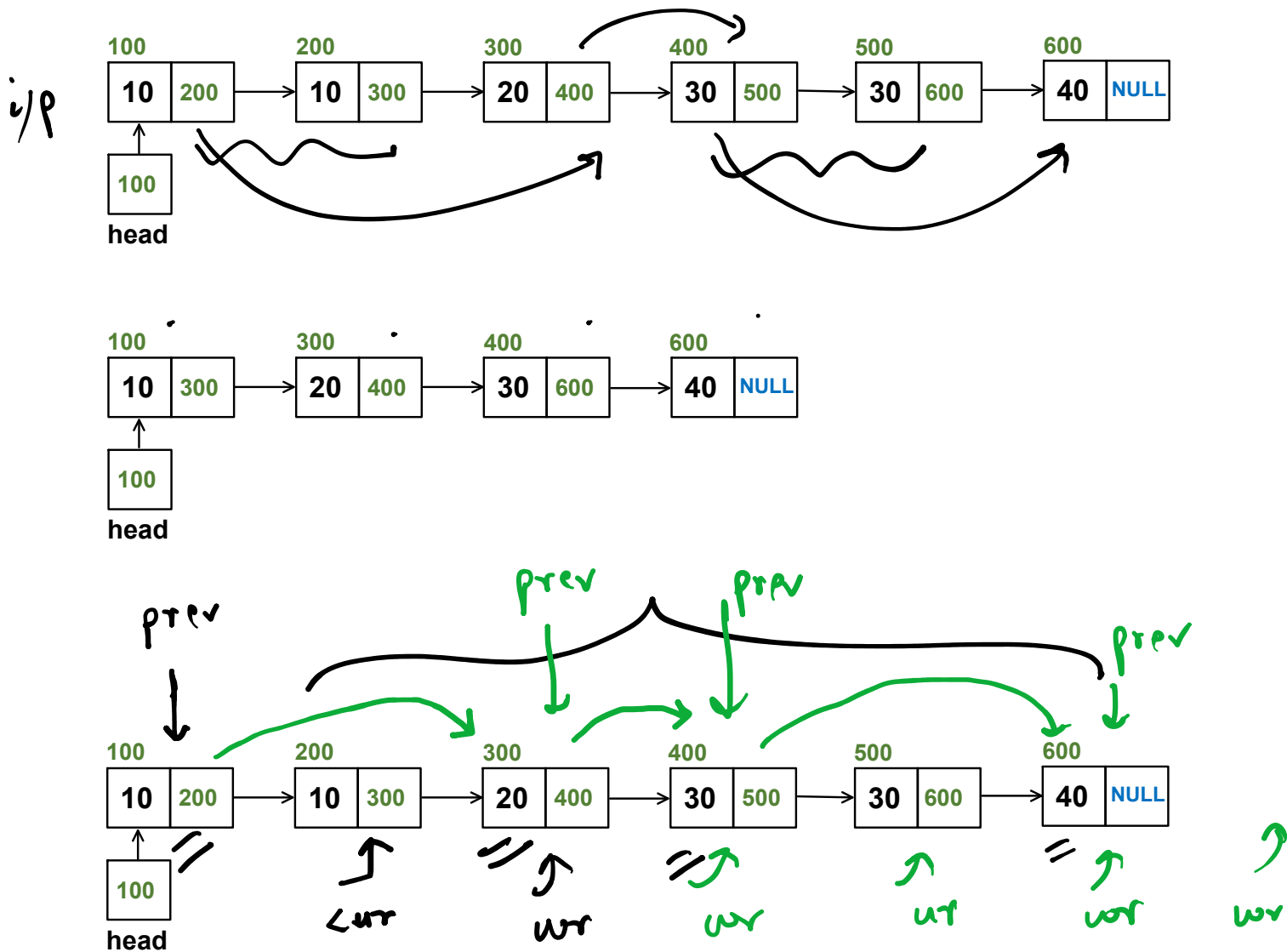
5 7 9

Sum of Linked List



Remove Duplicates from Sorted Linked List

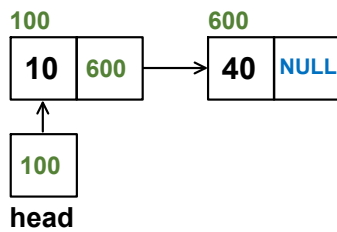
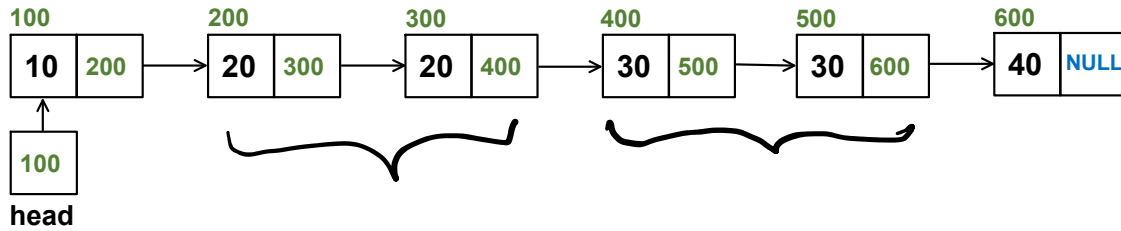
Remove Duplicates from Sorted Linked List I



[HW] Remove Duplicates from Sorted Linked List II

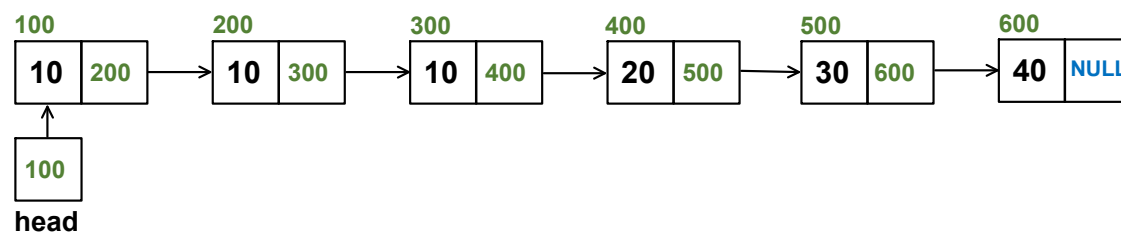
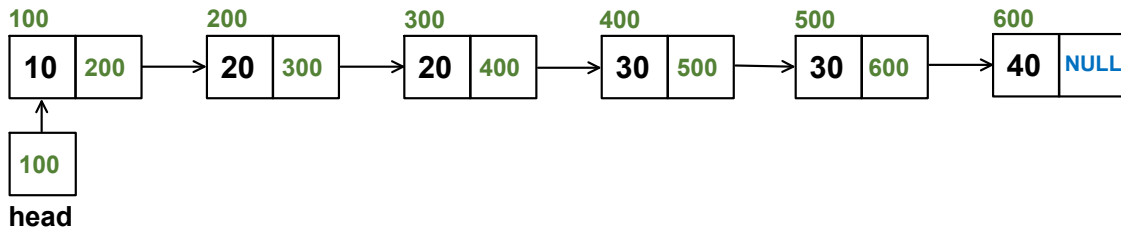
→ try to solve this
recursively
my iterative
head may change

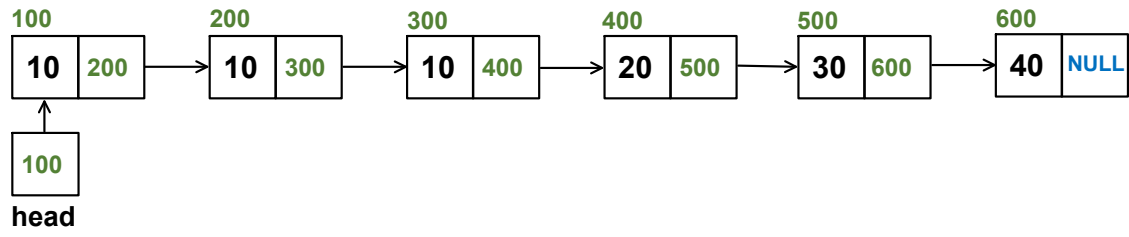
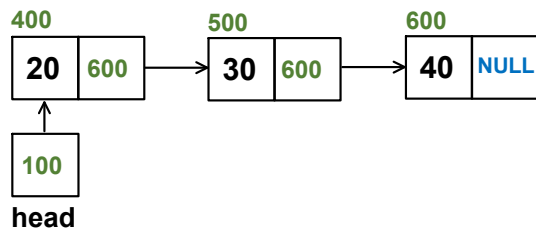
Remove Duplicates from Sorted Linked List II



10 → 10 → 10 → 20 → 30 → 40

20 → 30 → 40





Bubble Sort a Linked List

